CS3230 Design and Analysis of Algorithms	July 23, 2020

Solution *Sketches* to Problem Set 5

Name: Divesh Aggarwal

Instruction

This tutorial contains two kinds of questions - questions marked with an asterisk (*) are part of the assignment, and you need to submit your solutions to these questions; solutions to unmarked questions need not be submitted, but you should still attempt them. Your assignment solution will be graded. Submit your assignment solution by uploading to the LumiNUS. Solutions typeset using LaTeXare encouraged but not necessary. You may also submit scanned handwritten solutions, but they must be clearly readable. Solutions that are not readable may receive only partial marks. Late submissions will receive no marks. This assignment solution will be due by 11:59 pm, July 26, 2020 (Sunday).

Problem 5-1 (A polynomial time algorithm for 2SAT)

We want to show an efficient algorithm for 2SAT, which might be a little surprising since 3SAT is NP-hard. Given an instance ϕ of 2SAT, we first add a new literal z to each one-literal clause. We then construct a directed graph $G = (V, E)$ with vertices V labeled by the literals x and $\overline{x}$ for each variable x occuring in ϕ. This graph has an edge $(u, v) \in E$ directed from vertex u to vertex v if the clause $(\overline{u} \vee v)$ is in ϕ. If $(\overline{u} \vee v)$ is in ϕ, so is $(v \vee \overline{u})$ because of commutativity of $\vee$. Thus, if $(u, v) \in E$, then $(\overline{v}, \overline{u}) \in E$ as well (See Figure 1). Note that $(u, v) \neq (\overline{v}, \overline{u})$ because this requires that $v = \overline{u}$, which is not allowed. Let $u \neq \overline{w}$. It follows that if there is a path from u to w in G, there is a distinct path from $\overline{w}$ to $\overline{u}$) obtained by reversing the directions of each edge on the path and replacing the literals by their complements.

To understand why these edges are chosen, note that if all clauses of ϕ are satisfied and $(\overline{u} \vee v)$ is in ϕ, then $u = 1$ implies that $v = 1$. This implication relation, denoted $u \Rightarrow v$, is transitive. If there is a path $(u_1, u_2, \ldots, u_k)$ in G, then there are clauses $(\overline{u}_1 \vee u_2), (\overline{u}_2 \vee u_3), \ldots, (\overline{u}_{k-1} \vee u_k)$ in ϕ. If all clauses are satisfied and if the literal $u_1 = 1$, then each unnegated literal on this path must have value 1.

(a) Show that if there exists a variable x such that there is a path in G from x to $\overline{x}$ and from $\overline{x}$ to x, then ϕ is not satisfiable.

Solution: Suppose there is a variable x such that such paths exists. This means that $x \Rightarrow \overline{x}$ and $\overline{x} \Rightarrow x$ which is a logical contradiction. This implies that ϕ is unsatisfiable. $\square$

(b) Show that if ϕ is not satisfiable, then there must be a literal x such that there is a path from x to $\overline{x}$, and from $\overline{x}$ to x. Conclude that 2SAT can be solved in polynomial time.

Solution: Suppose there isn't such a literal. Then, we can get a satisfying assignment as follows.

Recursively do the following.

1. Pick an arbitrary literal ℓ that hasn't been assigned a value and for which there is no path from ℓ to $\overline{\ell}$. Assign $\ell = 1$.
2. Then use DFS to visit every vertex reachable from ℓ and assign 1 to all the corresponding literals.

First we show that we didn't assign a value to a variable that was already assigned a different value in a previous step. Suppose we assigned $\ell_1 = 1$ at a certain step, and then since there was a path from ℓ_1 to t, we assigned $t = 1$ and then we assigned $\ell_2 = 1$ (since it was not assigned a value) and then there was a path from ℓ_2 to $\overline{t}$, we assign $t = 0$. However, in this case, there is a path from t to $\overline{\ell}_2$, which means we already assigned a value to ℓ_2 during the DFS visit of ℓ_1. This shows that whenever our algorithm assigns a value to a variable, it must be consistent with prior assignments.

Finally, to conclude correctness, we need to show that the assignment obtained is consistent with all clauses. Suppose it is not consistent with some clause (u, v). This means both u and v were assigned 0, which is same as saying that vertices corresponding to both $\overline{u}$ and $\overline{v}$ were visited during the execution of the algorithm. Without loss of generality, assume $\overline{u}$ was visited before $\overline{u}$. Since (u, v) is a clause, the graph has an edge $(\overline{u}, v)$, and hence the algorithm assigns $v = 1$ contradicting our assumption. $\square$

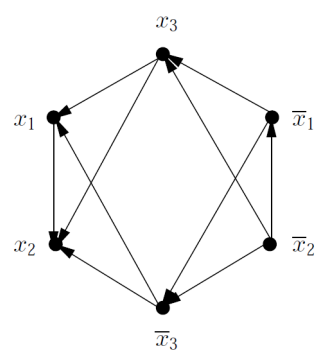

Figure 1: A graph capturing the implications associated with the following satisfiable instance of 2-SAT: $(x_3 \vee x_2) \wedge (x_3 \vee x_1) \wedge (\overline{x}_3 \vee x_2) \wedge (\overline{x}_1 \vee x_2) \wedge (\overline{x}_3 \vee x_1)$.

Problem 5-2 (*Avoiding Tolls) 15 Points + 3 Bonus Points

You are given a map $G = (V, E)$ with cities V connected by roads E. Each road (edge) is labeled with a weight which is a real number. You are located in city $s \in V$. You are also given an array A of boolean values that tells you if there is a toll on that road. More precisely, for road $e \in E$ we

have $A[e] = 1$ if and only if there is a toll on e. Being the low budget traveler that you are, your budget allows for at most one such toll to be payed for any given trip (path).

Let $d(s,t)$ denote the minimum possible sum of weights of a path from s to t for any $s, t \in V$. If there is no path from s to t, then $d(s,t) = \infty$, ~~and if there is a path from s to t that contains a negative cycle, then $d(s,t) = -\infty$ (since one can cycle along the path an arbitrary number of times).~~ You may assume that there is no negative cycle in this question.

Similarly, let $c(s,t)$ denote the minimum possible sum of weights of a path from s to t that passes through at most 1 toll.

(a) Construct a graph $G' = (V', E')$ and mappings $f : V \mapsto V'$, $g : V \mapsto V'$ such that $|V'| = 2|V|$, $|E'| \leq 2|E| + |V|$ and for any $s, t \in V$, $c(s,t) = d(f(s), g(t))$. Namely, you reduce the "constrained" problem on G to "unconstrained" problem in G'. ~~Remember to consider the case when $c(s,t)$ is $-\infty$, and~~ prove the correctness of your solution. (6 Points)

Solution: Let $V' = V_1 \cup V_2$, where V_1 and V_2 are two disjoint copies of V. For any $v \in V$, we denote the corresponding vertices in V_1 and V_2 as v_1 and v_2, respectively.

The edges in E' with corresponding edge weights is defined as follows. For all $v \in V$, we add an edge in E' with weight 0 from v_1 to v_2. In addition, for all $e = (u, v) \in E$,

- Add two edges (u_1, v_1) and (u_2, v_2) to E' with edge weight $W[u, v]$ if and only if $A[e] = 0$, i.e., if e is not a toll edge.
- Add an edge (u_1, v_2) to E' with edge weight $W[u, v]$ if and only if $A[e] = 1$, i.e., if e is a toll edge.

Thus, the number of edges in E' is at most $2|E| + |V|$. Furthermore, we define $f(v) = v_1$, and $g(v) = v_2$. Then for any path from vertex u to v in G that passes through 0 or 1 toll, there is a corresponding path in G' from u_1 to v_2 that has the same total weight. Also, for any path from u_1 to v_2 in G', there is a corresponding path from u to v in G that passes through at most one toll and has the same total weight. □

(b) Give an algorithm that takes as input s and finds a shortest path with at most one toll road from s to all cities in V. Analyze the running time of your algorithm. (1 Point)

Solution: Run Bellman-Ford Algorithm on G'. The running time is $O(nm + n^2)$. □

(c) Now assume your buddy Billybob who works at a major airlines company has given you a free plane ticket to any city in V, meaning you can start your trip at any node. As before, once you start your road trip, you are still refusing to pay more then a single toll on any such trip. To help plan the trip, your job is to give an algorithm to find a shortest path with at most one toll road between *all pairs* of cities in V. Analyze the running time of your algorithm. (2 Points)

Solution: Run Johnson's Algorithm on G'. The running time is $O(nm + n^2 \log n)$. □

(d) Here you will solve the problem in part (c) directly on graph G, without constructing the helper graph G'. Let $W = \{w(i,j)\}$ be the original edge weight matrix and $W' = \{w'(i,j)\}$ be the same edge matrix except we replace $w'(i,j) = \infty$ if $A(i,j) = 1$ (i.e., never use toll roads in W'). For simplicity, assume W' is pre-computed for you. For $0 \le k \le n$, let

- D^k be the matrix of all shortest distances w.r.t. W' which only use nodes $\le k$ as intermediate nodes.
- C^k be the matrix or all shortest distances w.r.t. W which only use nodes $\le k$ as intermediate nodes, but *also use at most one toll.*

Fill in the blanks below to directly modify the Floyd-Warshall algorithm to compute the correct answer for problem (c) in time $O(n^3)$. Argue the correctness of your algorithm. (6 Points)

FLOYD-WARSHALL(W, W')
  $n = W.rows$
  $D^0 = ...$
  $C^0 = ...$
  For $k = 1$ **to** n **Do**
    $C^k = (c^k_{i,j})$ be a new $n \times n$ matrix
    $D^k = (d^k_{i,j})$ be a new $n \times n$ matrix
    For $i = 1$ **to** n **Do**
      For $j = 1$ **to** n **Do**
        $d^k_{i,j} = \min(.....)$
        $c^k_{i,j} = \min(.....)$
  Return .....

Solution:

FLOYD-WARSHALL(W, W')
  $n = W.rows$
  $D^0 = W'$
  $C^0 = W$
  For $k = 1$ **to** n **Do**
    $C^k = (c^k_{i,j})$ be a new $n \times n$ matrix
    $D^k = (d^k_{i,j})$ be a new $n \times n$ matrix
    For $i = 1$ **to** n **Do**
      For $j = 1$ **to** n **Do**
        $d^k_{i,j} = \min\left(d^{k-1}_{i,k} + d^{k-1}_{k,j}, d^{k-1}_{i,j}\right)$
        $c^k_{i,j} = \min\left(d^{k-1}_{i,k} + c^{k-1}_{k,j}, c^{k-1}_{i,k} + d^{k-1}_{k,j}, c^{k-1}_{i,j}\right)$
  Return D^n

Notice that D^k is updated only using D^{k-1}, enforcing the constraint that distance computed in D^k is w.r.t. W' and never use toll roads at all.

Likewise, when we update C^k, we use exactly one value from C^{k-1}, contributing to the usage of at most one toll.

The rest, i.e., choosing intermediate node k, etc., follows as the proof of Floyd-Warshall. $\square$

Bonus: (e) Modify your solution in part (d) to solve the problem that allows at most 2 tolls. You only need to provide the recursive formula here, and you don't need to prove correctness. Either 0 or 3 Points is given for the question. (3 Points) (**Hint**: You should need 3 matrix instead of 2.)

Solution: D^k and C^k are as defined in part (d) and we further define B^k to be the matrix of all shortest distance which only uses nodes $\leq k$ as intermediate nodes, but uses at most two tolls. The recursive relation for $b^k_{i,j}$ is:

$$b^k_{i,j} = \min\left(d^{k-1}_{i,k} + b^{k-1}_{k,j}, b^{k-1}_{i,k} + d^{k-1}_{k,j}, c^{k-1}_{i,k} + c^{k-1}_{k,j}, b^{k-1}_{i,j}\right) .$$

$\square$

Problem 5-3 (*Don't put too much weight, plan B!) 5 Points

This is a slight modification of a question in Assignment 4! We consider an undirected graph in Assignment 4, now we will consider a directed graph.

Jack loves hiking and wants to always try different hiking trails. He has a map with n different landmarks labeled $1, \ldots, n$. Since there will be no water available during the paths between landmarks, the map also carries information about the amount of water $w[i, j]$ you need to carry to survive while taking the direct road from landmark i to j (without any intermediate landmarks in between), for $1 \leq i, j \leq n$.

On a given day, Jack may start at any landmark i and end at any landmark j for $1 \leq i, j \leq n$. He does not mind to visit some intermediate landmarks on the way, as long as the maximum weight that he needs to carry during the entire journey to be as little as possible. You may assume that there is plenty of water to fill from at each of the landmarks.

Assume you are given an **directed** edge-weighted graph $G = (V, E)$, where the set of vertices correspond to the n landmarks and the weight of the edge from i to j denotes the amount of water needed when taking the direct road from i to j. Being a directed graph, $w[i, j]$ might be different from $w[j, i]$ (think that going uphill might require much more water than going downhill).

Design an $O(n^3)$ algorithm to help Jack determine the smallest possible maximum weight $s[i, j]$ that he needs to carry when going from i to j, for all pairs of vertices i, j.

Solution: The algorithm is the same as Floyd-Warshall except the following change in the recurrence relation. The proof of correctness and running time is the same as Floyd-Warshall.

$$c^{(k)}_{ij} = \begin{cases} w[i,j], & \text{if } k = 0 \\ \min\left(c^{(k-1)}_{ij}, \max\left(c^{(k-1)}_{ik}, c^{(k-1)}_{kj}\right)\right), & \text{otherwise} \end{cases}$$

$\square$

Problem 5-4 (Arbitrage Cycle)

In this problem we will discover how you, yes **you**, can be employed by Wall Street and cause a major economic collapse! The arbitrage business is a money-making scheme that takes advantage of differences in currency exchange. In particular, suppose 1 US dollar buys 120 Japanese yen, 1 yen buys 0.01 euros, and 1 euro buys 1.2 US dollars. Then, a trader starting with \$1 can convert their money from dollars to yen, then from yen to euros, and finally from euros back to dollars, ending with \$1.44! The cycle of currencies \$ $\rightarrow$ ¥$\rightarrow$ €$\rightarrow$ \$ is called an **arbitrage cycle**. Of course, finding and exploiting arbitrage cycles before the prices are corrected requires extremely fast algorithms.

Suppose n different currencies are traded in your currency market. You are given the matrix $Exch[1\,..\,n, 1\,..\,n]$ of exchange rates between every pair of currencies; for each i and j, one unit of currency i can be traded for $Exch[i, j]$ units of currency j. (Do not assume that $Exch[i, j] \cdot Exch[j, i] = 1$.)

(a) Describe an algorithm that returns an array $MaxAmt[1\,..\,n]$, where $MaxAmt[i]$ is the maximum amount of currency i that you can obtain by trading, starting with one unit of currency 1, assuming there are no arbitrage cycles.

Solution:

We convert the problem of finding the maximum amount of currency obtained by trading starting from one unit of currency 1 to finding the shortest paths starting from vertex 1 as follows. We create a complete graph where $V = \{1, \ldots, n\}$ and weight $w(i \rightarrow j) = -\log Exch[i, j]$ for all $i, j \in V$. Notice that by the monotonicity property of logarithms and $\log a \cdot b = \log a + \log b$, we have that a shortest path from 1 to u of length ℓ correspond to the largest sequence of trades that starts with one unit of currency 1 and ends with $10^{-\ell}$ units of currency u, and vice versa.

Since the resulting weights may be negative and by the assumption that there are no arbitrage cycles which means there are no negative cycles, we can use BellmanFord to compute the shortest paths from vertex 1 to every other vertex. Let its output be $dist[1\,..\,n]$. We compute and output $MaxAmt[1\,..\,n]$ where $MaxAmt[i] = 10^{-dist[i]}$.

□

(b) Describe an algorithm to determine whether the given matrix of currency exchange rates creates an arbitrage cycle.

Solution:

We convert the problem as in (a) and run one more inner iteration of BellmanFord to detect negative cycles. Report an arbitrage cycle detected iff a negative cycle detected.

□

Problem 5-5 (On the Way to the Examination Hall)

John, who lives in a node s of a weighted undirected graph G (with non-negative weights), is getting late and has to reach the venue of his high school final exam at node h as soon as possible. However, he has to buy some pencils on his way to the examination hall. He can get pencils at any stationary, and the stationary shops form a subset of the vertices $B \subset V$. Thus, starting at s, he must go to some node $b \in B$ of his choice, and then head from b to h using the shortest total route possible (assume he wastes no time in the stationary). Help John to reach the examination hall as soon as possible, by solving the following sub-problems...

(a) Compute the shortest distance from s to all stationary shops $b \in B$.

Solution: To find the shortest distance from s to all stationary shops $b \in B$ we use a Dijkstra call on s. □

(b) Compute the shortest distance from every stationary shop $b \in B$ to h. Can one simply add a new "fake" source s' connected to all stationary shops with zero-weight edges and run Dijkstra from s'?

Solution: No, if we add a new "fake" source s' connected to all stationary shops with zero-weight edges and run Dijkstra from s' we are going to compute the shortest distance from any (not every) stationary $b \in B$ to h. As the graph is undirected, the easiest way to solve this problem is to run Dijkstra from h. □

(c) Combine parts (a) and (b) to solve the full problem.

Solution: The easiest way is to use a Dijkstra call on s and then run Dijkstra again from h [we use two calls to the Dijkstra's algorithm (one in part (a) and one in part (b)]. It remains to iterate through all $b \in B$ and find:

$$\min_b dist(s, b) + dist(b, h) \ .$$

□

(d) Your solution in part (c) used two calls to the Dijkstra's algorithm (one in part (a) and one in part (b)). Define a new graph G' on at most $2n$ vertices and at most $2m + n$ edges (and "appropriate" weights on these edges), so that the original problem can be solved using a *single* Dijkstra call on G'.

Solution: Similar to the Toll Problem, we define the following graph $G' = (V', E')$ on $2n$ vertices. The vertex set of V' of G' is $V' = V_1 \cup V_2$, where V_1 and V_2 are two disjoint copies of V. For any vertex $v \in V$, we use v_1 and v_2 to denote the corresponding vertex in V_1 and V_2 respectively.

For each edge $(u, v) \in E$, we add the edges (u_1, v_1) and (u_2, v_2) of the same weight to E'. Lastly, for each $b \in B$, we add the edge (b_1, b_2) of weight 0 to E'. Then, the original problem can be solved using a *single* Dijkstra call on G' starting from s_1. □

Problem 5-6 (Modified Dijkstra - Too Slow?)

Dijkstra's algorithm solves the single-source shortest-path problem on a weighted directed graph $G = (V, E)$, when all edge weights are non-negative. Suppose we wish to modify the algorithm so that it works on graphs which has negative weight edges as long as there is no *negative cycle*. Consider the following modified Dijkstra's algorithm.

```
ModifiedDijkstra(G, w, s)
1.  Initialize-Single-Source(G, s)
2.  S = ∅
3.  Q = V
4.  While Q ≠ ∅ Do
5.      u = Extract-Min(Q)
6.      S = S ∪ {u}
7.      For all edges u → v
8.          If dist(v) > dist(u) + w(u, v)
9.              dist(v) = dist(u) + w(u, v)
10.             If v ∈ Q, Then Decrease-Key(Q, v, dist(v))
11.             Else Insert(Q, v) and S = S \ {v}.
```

(a) Note that the only step where the above algorithm differs from the original Dijkstra algorithm is in Step 11. Give an example with the smallest possible number of vertices to show that if we remove step 11 from ModifiedDijkstra, then it does not solve the single-source shortest-path problem if the edge weights may be negative, even if there are no negative weight cycle.

Solution: Consider a graph with four vertices a, b, c, d and four edges $(a, b), (a, c), (c, b), (b, d)$ with edge weights, $w(a, b) = 1$, $w(a, c) = 2$, $w(c, b) = -2$, and $w(b, d) = 0$. Dijkstra algorithm, if executed on this graph with source vertex a will erroneously compute the shortest distance from a to d to be 1. □

(b) Assume that the input graph G has no negative weight cycle, although there may be some edges with negative weight. Show that the total number of times the value of $dist(v)$ changes in the above algorithm is finite. Hence argue that the algorithm terminates in a finite number of steps.

Solution: Given that there is no negative cycle, for any vertex v, $dist(v)$ is well-defined and finite. Each time $dist(v)$ decreases, its value drops by at least the smallest difference between any two edges, which is a constant. Hence, it can decrease only a finite number of times before it reaches the correct value.

Since we insert vertex into Q only after $dist(v)$ decreases, we insert a finite number of vertices into Q and the algorithm therefore would empty Q and terminate after a finite iterations □

(c) Notice that for all $v \in V$, the value of $dist(v)$ is at least the shortest distance from s to v in an execution of the above algorithm. Argue that when the algorithm terminates, for all $v \in V$, the value of $dist(v)$ is equal to the shortest distance from s to v. Hence conclude the

above algorithm correctly solves the single-source shortest path problem even for graphs with negative weight, as long as there is no negative weight cycle. Explicitly state where you need step 11 of the above algorithm in your proof.

Solution: Assume towards contradiction that $dist(v)$ is not correct upon termination. Let $s, u_1, \ldots, u_k, v$ be the shortest path from s to v and $dist(v)$ is greater than the total weight of this path. Let u_i $(i \leq k)$ be the last vertex on this path for which $dist(u_i)$ is equal to the total weight of the path $s, u_1, \ldots, u_i$. Then, at some point during the execution of the algorithm $dist(u_i)$ was assigned the correct value in Step 9. Step 11 ensures that u_i was guaranteed to be in Q after getting the correct value. Since the algorithm terminates, u_i should be dequeued at some point and u_{i+1} should receive that correct value as we relax all neighbours of u_i, which contradicts the fact that u_i was the last vertex on the path for which $dist(u_i)$ is equal to the total weight of the path $s, u_1, \ldots, u_i$. □

(d) Consider an example of a graph G with vertices $(s, v_1, \ldots, v_n)$ and edge weights $w(s, v_i) = 0$, and $w(v_i, v_j) = -2^{-i}$ for all $1 \leq i < j \leq n$. By finding an appropriate recurrence relation, show that ModifiedDijkstra takes $\Omega(2^n)$ time in the worst case, when executed on (G, s, w).

Solution: As usual, the algorithm begins by dequeuing s, then relaxing each $dist(v_i)$ to 0 for $v_1, \ldots, v_n$. This results in n 0-weight keys, all equal priority in the priority queue.

Assume that among vertices with equal priority, the algorithm dequeues the vertices in reverse chronological order.

Let $T(n)$ be the total number of times the relax operation is called for the n vertices $v_1, \ldots, v_n$. Clearly, since all edge weights are non-positive, v_1 is dequeued only when all $v_2, \ldots, v_n$ have been assigned the shortest distance ignoring the paths that use v_1. This takes at least $T(n-1)$ relax operations.

Since,

$$-2^{-1} < -2^{-2} - 2^{-3} - \cdots 2^{-n} ,$$

when the vertex v_1 is dequeued, its outgoing edges have lower weights than their previously discovered shortest paths. Thus, all vertices $v_2, \ldots, v_n$ are inserted back to Q and their weights set to $-1/2$. This takes $n-1$ relax operations. After this the algorithm again needs to perform $T(n-1)$ relax operations to assign the correct shortest distance to $v_2, \ldots, v_n$. Thus, the total number of relax operations in an execution of this algorithm is given by

$$T(n) = 2T(n-1) + n - 1 ,$$

which implies $T(n) = \Omega(2^n)$. □

Problem 5-7 (*NP hardness of Dominating Set) 10 Points

The dominating set of an undirected graph $G = (V, E)$, is defined to be a set of vertices $V' \subseteq V$ such that for any vertex $v \in V$, either $v \in V'$ or there exists $u \in V'$ such that $(u, v) \in E$. The dominating set problem asks the following: Given a graph $G = (V, E)$, and an integer $k > 0$, does there exist a dominating set of size at most k. In this problem, we will show the NP hardness

of Dominating Set by giving a reduction from the Vertex Cover problem. Given an undirected graph $G^* = (V^*, E^*)$, we define a graph G by *adding* vertices and edges to G^* to create a graph $G = (V, E)$ as follows. We initialize $V = V^*$ and $E = E^*$. Then, for every edge $(x, y) \in E^*$, we create a new vertex $v_{x,y}$ and add $v_{x,y}$ to V and edges $(x, v_{x,y})$ and $(y, v_{x,y})$ to E. We now want to prove the following assertion:

Assertion: G^* has a vertex cover of size k if and only if G has a dominating set of size k'.

(a) We say that a vertex v in a graph is isolated if there is no edge in the graph incident on the vertex v. Let $|V^*| = n$, and let m be the number of isolated vertices in V^*. Give a value of k' (without proof) as a function of k, n, m such that the above assertion is true. (3 Points)

Solution: $k' = \min(k + m, n)$. (You won't be penalised for stating only $k' = k + m$.) □

(b) Show that if G^* has a vertex cover of size k, then G has a dominating set of size k' for your choice of k' in part (a). (3 Points)

Solution: Let $v_1, \ldots, v_k$ be a vertex cover of G^*, and $u_1, \ldots, u_m$ be the m isolated vertices in V^*. We claim that $V_1 = \{v_1, \ldots, v_k, u_1, \ldots, u_m\}$ is a dominating set in G.

To see this, consider any vertex $v \notin V_1$. Then, it cannot be an isolated vertex. If $v \in V^*$, then there is an i such that $(v_i, v) \in E^* \subseteq E$ (since $v_1, \ldots, v_k$ is a vertex cover of G^*, the other endpoint of any edge incident to v must be in the vertex cover). Thus, v is dominated by V_1. If $v \notin V^*$, then $v = v_{x,y}$ for some edge (x, y) in V^*. Then, one of x or y is in $\{v_1, \ldots, v_k\}$, since it is a vertex cover, and hence $v_{x,y}$ is dominated by V_1. □

(c) Show that if G has a dominating set of size k' for your choice of k' in part (a), then G^* has a vertex cover of size k. (4 Points)

Solution: Without loss of generality, we may assume that the dominating set of G does not contain any vertex of the form $v_{x,y}$ (i.e., it only contains vertices from V^*). This is because if there is a dominating set containing $v_{x,y}$, then since $v_{x,y}, x, y$ form a clique and there is no other adjacent vertices to $v_{x,y}$, we can replace $v_{x,y}$ by any of x, y to get another dominating set of the same size.

Also, a dominating set must contain all isolated vertices of V^*, which are also isolated vertices of V.

Let $v_1, \ldots, v_k$ be all the non-isolated vertices in the dominating set. We claim that $(v_1, \ldots, v_k)$ forms a vertex cover in V^*. To see this, consider an edge x, y, where none of x, y is in the set $\{v_1, \ldots, v_k\}$. Then, the vertex $v_{x,y}$ is not dominated by the dominating set, which is a contradiction. □

Problem 5-8 (*Majority 3SAT) 10 Points

Consider the following variant of 3SAT, called Majority3SAT. Just like 3SAT, the input to Majority3SAT is a boolean formula Φ in conjunctive normal form, with exactly three literals per clause.

Majority3SAT asks whether there is an assignment to the variables of Φ, such that every clause contains at least two TRUE literals.

Either "reduce Majority3SAT to 2SAT and conclude that Majority3SAT is in **P**" or "reduce 3SAT to Majority3SAT and conclude that Majority3SAT is **NP-Complete**".

Solution: We will show how to reduce Majority3SAT to 2SAT. Let the boolean formula Φ be an input instance of Majority3SAT in conjunctive normal form. For each clause $(l_1, l_2, l_3) \in \Phi$, we construct the following 2SAT clauses: $(l_1 \vee l_2), (l_2 \vee l_3), (l_1 \vee l_3)$. We connect all the constructed clauses in CNF form to be the 2SAT boolean formula Ψ. It remains to show that Φ is majority satisfiable if and only if Ψ satisfiable.

In fact, any majority satisfying assignment a to Φ is also a satisfying assignment to Ψ and vice versa. If an assignment a majority satisfies a clause $(l_1, l_2, l_3) \in \Phi$, then at least two of the literals l_1, l_2, l_3 are TRUE. Hence, all three corresponding clauses in Ψ: $(l_1 \vee l_2), (l_2 \vee l_3), (l_1 \vee l_3)$ are satisfied by a. It is easy to check that the other direction is also true.

We only increase the number of clauses by a factor of 3. Hence, this is a valid reduction from Majority3SAT to 2SAT. $\square$

Problem 5-9 (Hardness of MAX-2SAT and MAX-CUT)

(a) We want to analyze a reduction from 3SAT to the following language:

$$\mathsf{MAX\text{-}2SAT} = \{(\phi', k) \mid \phi' \text{ is a 2-CNF formula, and there is an assignment that satisfies at least } k \text{ clauses}\}.$$

Our reduction is the following: Given a 3SAT instance ϕ, we will output a MAX-2SAT instance (ϕ', k), where ϕ' is a 2-CNF formula. To construct ϕ', do the following: for each clause $(x \vee y \vee z)$ in ϕ, add the following 10 clauses to ϕ' (where w is a fresh variable for each clause):

$$(x), (y), (z), (\neg x \vee \neg y), (\neg y \vee \neg z), (\neg x \vee \neg z), (w), (x \vee \neg w), (y \vee \neg w), (z \vee \neg w)$$

Find a value of k such that $(\phi', k) \in \mathsf{MAX\text{-}2SAT}$ if and only if $\phi \in \mathsf{3SAT}$. Prove the correctness of the reduction.

Solution: Consider a certain clause $(x \vee y \vee z)$ in ϕ.

If one of x, y, z is satisfied (wlog x), then setting $w = \mathsf{false}$, we get $(x), (\neg x \vee \neg y), (\neg y \vee \neg z), (\neg x \vee \neg z), (x \vee \neg w), (y \vee \neg w), (z \vee \neg w)$ are satisfied.

If two of x, y, z are satisfied (wlog x, y), then setting $w = \mathsf{false}$, we get $(x), (y), (\neg y \vee \neg z), (\neg x \vee \neg z), (x \vee \neg w), (y \vee \neg w), (z \vee \neg w)$ are satisfied.

If all of x, y, z are satisfied, then setting $w = \mathsf{true}$, we get $(x), (y), (z), (w), (x \vee \neg w), (y \vee \neg w), (z \vee \neg w)$ are satisfied.

Now if none of x, y, z are satisfied, then setting $w = \mathsf{true}$, we only get $(\neg x \vee \neg y), (\neg y \vee \neg z), (\neg x \vee \neg z), (w)$ are satisfied; and setting $w = \mathsf{false}$, we only get $(\neg x \vee \neg y), (\neg y \vee \neg z), (\neg x \vee \neg z), (x \vee \neg w), (y \vee \neg w), (z \vee \neg w)$ are satisfied.

Therefore, if $(x \vee y \vee z) = 1$ then 7 clauses of ϕ' are satisfiable, and if $(x \vee y \vee z) = 0$ then at most 6 clauses of ϕ' are satisfiable. Thus, we set $k = 7m$. $\square$

Now consider the following language:

$$\mathsf{MAX\text{-}CUT} = \{(G, c) \mid G \text{ is a multigraph with a cut of size at least } c\}.$$

A *cut* in a graph is a partition of its vertices into two parts. The size of the cut is the number of edges which "cross" the cut (whose endpoints are in opposite parts). A multigraph means we allow duplicate edges.

We now analyze a reduction from MAX-2SAT to MAX-CUT. Given an instance (ϕ, k) of MAX-2SAT, let n be the number of variables occuring in ϕ, and m the number of clauses. Consider the following graph: G_ϕ is a graph with vertices labeled x_i and $\neg x_i$ for each variable x occuring in ϕ, and two special vertices labeled T and F. We add $5m$ edges between T and F, and $5m$ edges between each pair $(x_i, \neg x_i)$ — see Figure 2. Then, for each clause $(x \vee y) \in \phi$, where x and y are literals, we add the following 7 edges (see clause subgraph in Figure 3): (x, y), (T, x), (T, y), and two copies of the edges (x, F) and (y, F).

(b) Show that in the largest cut in G_ϕ, T and F must be in opposite parts.

Solution: If T and F are in the same part, separating them can only increase the size of the cut by at least $5m - 4m = m$. □

(c) Show that in the largest cut in G_ϕ, the vertices corresponding to x and $\neg x$ must be in opposite parts.

Solution: If x and $\neg x$ are in the same part, separating them can only increase the size of the cut by at least $5m - 4m = m$. □

(d) Argue that $(\phi, k) \in$ MAX-2SAT if and only if $(G_\phi,\ 5m + 5mn + 4k + 2(m - k)) \in$ MAX-CUT.

Solution: First we argue that if $(\phi, k) \in$ MAX-2SAT then $(G_\phi,\ 5m + 5mn + 4k + 2(m - k)) \in$ MAX-CUT.

Suppose ϕ has an assignment satisfying $\geq k$ clauses. Create the following cut (V_T, V_F): put T in V_T and put F in V_F; for all $i \in [n]$, if x_i is true then put x_i in V_T, $\neg x_i$ in V_F; but if x_i is false then put x_i in V_F, $\neg x_i$ in V_T.

Since we separate T, F and x_i, $\neg x_i \, \forall i$, this gives $5m + 5mn$ edges to the cut. Let (x, y) be a satisfied clause. Either both were set true or just one of them, that clause gives 4 edges to the cut. Since there are k satisfied clauses, they give $4k$ edges to the cut. Now let (x, y) be an unsatisfied clause. Both were set false, so that clause gives 2 edges to the cut. Since there are $m - k$ satisfied clauses, they give $2(m - k)$ edges to the cut.

Next we argue that if $(G_\phi,\ 5m + 5mn + 4k + 2(m - k)) \in$ MAX-CUT then $(\phi, k) \in$ MAX-2SAT.

Suppose G_ϕ has a cut of size $5m + 5mn + 4k + 2(m - k)$. By (b) and (c), $5m + 5mn$ edges of the cut account for separated T, F and x_i, $\neg x_i \, \forall i$. We set the literals that belong to the same part as T to true, and the ones with F as false. The $4k$ edges account for either having both or one of the literal vertices in the same part with T for each k clause subgraph, which give k satisfied clauses; and the $2(m - k)$ edges account for having both literal vertices in the same part with F for each $m - k$ clause subgraph, which give $m - k$ unsatisfied clauses.

□

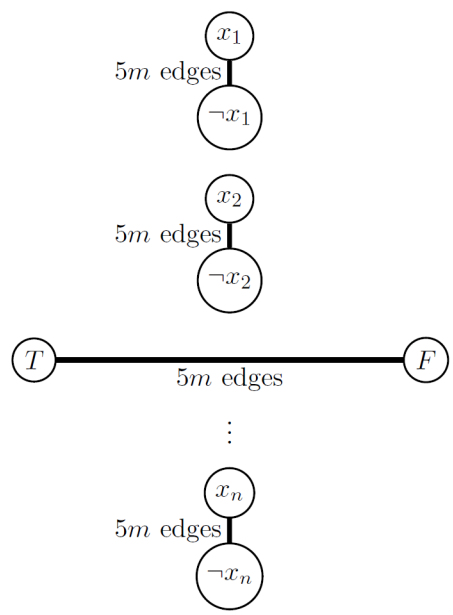

Figure 2: Starting graph for G_ϕ, where ϕ has n variables, $x_1, \ldots, x_n$.

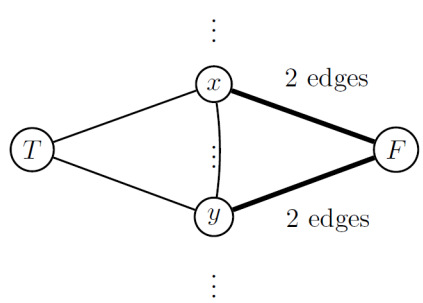

Figure 3: Edges to add for a clause of the form $(x \vee y)$.